# INDUSTRIAL LOAD FORECASTING AND CONTRACT DEMAND ALERT SYSTEM

A Minor Project Report
Submitted in partial fulfilment of requirement of the
Degree of
**BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE & ENGINEERING**

**BY**

**Ashika Jain**
**Aryansh Kshetri**

Under the Guidance of

**Dr. Garima Silakari Tukra**
**Prof. Digendra Singh Rathore**


**Department of Computer Science & Engineering**
**Faculty of Engineering**
**MEDICAPS UNIVERSITY, INDORE- 453331**

**APRIL 2026**

# Report Approval

The project work "**Industrial Load Forecasting and Contract Demand Alert System**" is hereby approved as a creditable study of an engineering/computer application subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

**Internal Examiner**
Name: _____
Designation: _____
Affiliation: _____

**External Examiner**
Name: _____
Designation: _____
Affiliation: _____

# Declaration

We hereby declare that the project entitled "**Industrial Load Forecasting and Contract Demand Alert System**" submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering, completed under the supervision of Dr. Garima Silakari Tukra , Prof. Digendra Singh Rathore.

Further, we declare that the content of this project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

**Signature and Name of the Student(s) with Date:**

_____ Date: _____

Ashika Jain

_____ Date: _____

Aryansh Kshetri

# Certificate

I/We, **Dr. Garima Silakari Tukra , Prof. Digendra Singh Rathore** certify that the project entitled "**Industrial Load Forecasting and Contract Demand Alert System**" submitted in partial fulfilment for the award of the degree of Bachelor of Technology in Computer Science & Engineering by **Ashika Jain** and **Aryansh Kshetri** is the record carried out by them under my/our guidance and supervision.

The work presented is genuine and has not been submitted to any other Institute or University for any degree or diploma.

_____
Dr. Garima Silakari Tukra
Medicaps University, Indore

_____
[Name of External Guide (If any)]
Name of the Organization

_____
Prof. Digendra Singh Rathore
Medicaps University, Indore

_____
Dr. Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
Medicaps University, Indore

# Acknowledgement

We would like to express our sincere gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided us with every facility to successfully carry out this project, and our profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, Medicaps University, for unfailing support and encouragement. We also thank **Dr. Ratnesh Litoria**, Dean, Faculty of Engineering, Medicaps University. We are deeply thankful to our project guide, Dr. Garima Silakari Tukra, Prof. Digendra Singh Rathore, Department of Computer Science & Engineering, for consistent technical direction, patience, and mentorship throughout the development and documentation of this project. We also thank **Dr. Kailash Chandra Bandhu**, Head of the Department of Computer Science & Engineering, for creating an environment where applied, practical projects are genuinely valued and supported.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Ashika Jain
Aryansh Kshetri
B.Tech. III Year
Department of Computer Science & Engineering
Faculty of Engineering
Medicaps University, Indore

# Abstract

Factories operating under contracted demand agreements with electricity providers face financial penalties when their electricity load exceeds the contracted threshold. The penalty is calculated on peak demand rather than average consumption, meaning a single hour of overrun in a billing cycle is enough to generate a significant charge. Most small and mid-sized industrial units have no way to anticipate this in advance — they discover violations only after the bill arrives.

This project develops a web-based Industrial Load Forecasting and Contract Demand Alert System that addresses this gap. The system accepts historical hourly electricity load data uploaded as a CSV file, applies a trained Random Forest Regression model to generate next-day (24-hour) and next-week (168-hour) load forecasts, and displays a real-time Safe or Risk alert depending on whether the predicted load is likely to exceed the user-defined contract demand limit.

The machine learning model is trained on one year of synthetic hourly factory data. Ten features are engineered from raw datetime and load values: hour of day, day of week, month, weekend flag, day of year, lag values at 1, 24, and 168 hours, and rolling averages over 3 and 24 hours. On a time-ordered held-out test set of 1,719 observations, the model achieves a Mean Absolute Error of 15.44 kW, Root Mean Square Error of 19.44 kW, and Mean Absolute Percentage Error of 8.09%, corresponding to approximately 92% prediction accuracy.

The system is implemented as a full-stack web application. The frontend is built with React and Next.js, deployed on Vercel. The backend is a FastAPI Python service deployed on Railway. Prediction results are stored per user in a cloud PostgreSQL database on Neon. User authentication is managed by Clerk. Security measures include rate limiting via SlowAPI, strict input validation, CORS enforcement, and environment variable management for all credentials. Users can save their factory settings including contract demand, which auto-fills the dashboard on subsequent logins.

**Keywords:** Electricity Load Forecasting, Contract Demand, Random Forest, FastAPI, Next.js, Machine Learning, Alert System, Time Series, Industrial IoT, Penalty Avoidance.

# Table of Contents

# List of Figures

# List of Tables

# Abbreviations

| Abbreviation | Full Form |
|---|---|
| API | Application Programming Interface |
| CORS | Cross-Origin Resource Sharing |
| CSV | Comma-Separated Values |
| DB | Database |
| DFD | Data Flow Diagram |
| ER | Entity Relationship |
| HTTP/HTTPS | Hypertext Transfer Protocol / Secure |
| IDE | Integrated Development Environment |
| JSON | JavaScript Object Notation |
| JSONB | Binary JSON (PostgreSQL column type) |
| JWT | JSON Web Token |
| kW | Kilowatt |
| kVA | Kilovolt-Ampere |
| MAE | Mean Absolute Error |
| MAPE | Mean Absolute Percentage Error |
| ML | Machine Learning |
| OOP | Object-Oriented Programming |
| pkl | Python Pickle serialized model file |
| REST | Representational State Transfer |
| RMSE | Root Mean Square Error |
| SCADA | Supervisory Control and Data Acquisition |
| SQL | Structured Query Language |
| UI | User Interface |
| UUID | Universally Unique Identifier |

# Notations & Symbols

| Symbol / Notation | Meaning |
|---|---|
| $y_t$ | Actual load value at time t (kW) |
| $\hat{y}_t$ | Predicted load value at time t (kW) |
| n | Total number of observations in the test set |
| t | Time index in hours |
| lag_kh | Load value k hours before the prediction point |
| roll_kh | Rolling average of load over k hours before the prediction point |
| D | User-defined contract demand threshold (kW) |
| $\hat{y}_{24}$ | Average predicted load over the next 24 hours (kW) |
| $\hat{y}_{168}$ | Vector of 168 hourly predicted load values (next 7 days) |
| RF | Random Forest Regressor model object |
| σ | Standard deviation (used in synthetic data noise generation) |
| → | Maps to / results in / transitions to |
| % | Percentage |

# Chapter 1
# Introduction

## 1.1 Introduction

Electricity is one of the largest operating costs for industrial manufacturing units. Most factories run under a contracted demand agreement that caps their maximum instantaneous consumption in kilowatts or kilovolt-amperes. Staying within this cap is financially critical. When actual consumption crosses the threshold — even for a single hour — the utility company applies a penalty charge calculated on the peak demand reading, not on average consumption. In India, where state electricity regulatory commissions govern these agreements, penalty rates are high enough that a single violation can add a meaningful cost burden to a plant's monthly bill.

The root cause of most violations is not deliberate overuse but a lack of forward visibility. Plant managers have historical data from smart meters, SCADA logs, or billing records, but this data is reviewed after the fact. By the time a violation appears on the bill, the chance to prevent it has passed.

Machine learning forecasting addresses this gap. Given a year or more of hourly load data, a regression model can learn the temporal patterns in a factory's consumption — morning ramp-ups, shift changes, weekend shutdowns, seasonal loads — and project those patterns 24 hours or 7 days forward. When the forecast signals a likely violation, the operator has time to defer a machine startup, reschedule a production batch, or activate power reduction measures.

This project builds a web-based Industrial Load Forecasting and Contract Demand Alert System. The system accepts historical load data as a CSV file, generates next-day and next-week predictions using a trained Random Forest model, and displays a Safe or Risk alert against the user's contract demand limit. No hardware integration is required. A factory operator with a browser and a spreadsheet of meter readings can access full forecasting capability.

## 1.2 Literature Review

Short-term electricity load forecasting has a substantial research history covering classical statistical methods, machine learning approaches, and hybrid systems.

Box and Jenkins [1] established ARIMA models as a standard for time-series forecasting. These perform well on regular, linear seasonal data. Feinberg and Genethliou [2] survey their application in power systems and note their limitations on industrial load, which is non-linear and schedule-driven. Exponential smoothing faces the same constraint.

Tree-based ensemble methods improved the picture significantly. Tso and Yau [3] compared regression, decision trees, and neural networks on building energy datasets and found that tree-based methods generalized better to industrial load patterns. Breiman [4] introduced Random Forests — ensembles of decision trees trained on bootstrap samples with averaged predictions — which reduce variance substantially compared to single trees and handle high-dimensional feature spaces without scaling requirements.

Feature engineering matters as much as model choice. Ahmad et al. [5] reviewed machine learning methods for building energy forecasting and established that lag features at 1-hour, 24-hour, and 168-hour intervals reliably capture daily and weekly periodicity. Qiu et al. [6] showed that combining time-based features with rolling averages and lag values outperforms any single feature category alone.

On the application side, Marinakis et al. [7] built a decision support system combining load forecasting with configurable alert thresholds and showed that predictive alerts reduced peak demand penalties by up to 18% in a pilot industrial study. Their architecture separates the forecasting engine from the user interface, the same separation adopted here.

**Table 1.1: Comparison of Existing Load Forecasting Approaches**

| Method | Accuracy on Industrial Data | Deployment Complexity | Key Limitation |
|---|---|---|---|
| ARIMA | Moderate | Low | Linear patterns only; fails on schedule-driven load |
| Decision Tree | Good | Low | Prone to overfitting on small datasets |
| Random Forest | High | Medium | Slower iterative inference at 168 steps |
| Neural Network | Very High | High | Requires large data; high compute cost |
| SVM Regression | Good | Medium | Struggles with high-dimensional lag features |

| | | | |
|---|---|---|---|
| This System (RF) | ~92% | Medium | Trained on synthetic data; no SCADA integration |

## 1.3 Objectives

1. To train a Random Forest Regression model that predicts industrial electricity load for the next 24 hours and next 7 days from historical hourly data with at least 90% accuracy.
2. To build a FastAPI backend that accepts CSV uploads, validates and cleans input data, engineers 10 features, runs the model iteratively, and returns structured prediction results.
3. To design a React and Next.js frontend dashboard that displays predictions through charts and KPI cards and shows a Safe or Risk alert against the contract demand threshold.
4. To integrate user authentication via Clerk so each factory account has isolated prediction history and saved settings.
5. To store all prediction results per user in a cloud PostgreSQL database (Neon) and surface them in a recent predictions table.
6. To allow users to save factory configuration including contract demand, which auto-fills on subsequent logins.
7. To deploy the complete system publicly on Vercel (frontend) and Railway (backend).
8. To enforce security: rate limiting, input validation, CORS policy, and environment variable management.

## 1.4 Significance

Contract demand penalties represent a recurring, avoidable cost that most small and mid-sized factories absorb without realising the loss could be prevented. A 2020 study by Marinakis et al. [7] showed that predictive alerting reduced peak demand penalties by up to 18% in a pilot facility. For a factory paying INR 50,000 per month in demand penalties, even a 10% reduction translates to INR 60,000 in annual savings.

Beyond the financial case, this project demonstrates that industrial load forecasting does not require expensive hardware, SCADA integration, or enterprise software subscriptions. A software-only, browser-accessible tool that works from a standard CSV upload makes this capability available to small and mid-sized factories that could never justify a commercial energy management platform.

From an academic standpoint, the project contributes a complete, documented implementation of a feature-engineered Random Forest forecasting pipeline with a publicly deployed API

backend — covering not just model accuracy but security design, multi-user authentication, database schema, and cloud deployment architecture.

## 1.5 Research Design

The project follows an iterative and incremental development model. Five phases were completed sequentially, each producing a working, tested increment before the next phase began: (1) synthetic data generation and ML model training, (2) FastAPI backend development with security hardening, (3) React and Next.js frontend dashboard development, (4) frontend-backend integration with Neon database connection, and (5) Clerk authentication, factory settings management, and deployment.

The ML model evaluation uses a strictly time-ordered 80/20 train-test split with no random shuffling. This is the correct methodology for time-series forecasting because shuffling would leak future information into the training set and produce artificially optimistic accuracy metrics.

## 1.6 Source of Data

Real factory load data was not available during development. A synthetic dataset was generated in Python to simulate one year of hourly electricity consumption, producing 8,760 rows. Each row contains a datetime in YYYY-MM-DD HH:MM:SS format and a load value in kilowatts.

The generation logic reflects realistic factory behavior: weekday production hours (9 AM to 5 PM) produce load peaks of 350–400 kW with a morning ramp from 6 AM; night hours carry a base load of approximately 200 kW; weekends run at 150–165 kW; monthly seasonal variation adds ±40 kW driven by a sinusoidal term; Gaussian noise with $\sigma = 15$ kW is applied to prevent the model from fitting a perfectly deterministic pattern. The system accepts real factory data in the same CSV format when available.

## 1.7 Chapter Scheme

Chapter 2 specifies the system requirements: user characteristics, functional requirements, dependencies, performance, hardware, and constraints. Chapter 3 covers the complete design: algorithm, function-oriented design, all six system diagrams, and database design. Chapter 4 describes implementation, testing, and installation. Chapter 5 presents results and discussions including interface representations, module descriptions, and database snapshots. Chapter 6 states the summary and conclusions. Chapter 7 outlines the future scope.

# Chapter 2
## Requirements Specification

### 2.1 User Characteristics

Three user classes interact with the system:

Factory Operator — The primary user. Uploads CSV data, sets contract demand, triggers predictions, and reads the Safe or Risk alert. No technical background is assumed. The interface is designed to be self-explanatory without training.

Factory Manager — Reviews prediction history and adjusts contract demand settings. Makes operational decisions based on alert output. May also run predictions directly.

System Administrator — Manages cloud deployment, environment variable configuration, and database maintenance. Technical background required. Not the intended end user of the forecasting dashboard.

### 2.2 Functional Requirements

**Table 2.1: Functional Requirements**

| Req. ID | Requirement Description | Priority |
|---|---|---|
| FR-01 | System shall accept CSV file uploads containing datetime and load_kw columns. | High |
| FR-02 | System shall validate: .csv extension only, max 5 MB, 168–10,000 rows, numeric load values. | High |
| FR-03 | System shall clean data: fix datetime parsing errors, remove rows with load ≤ 0, drop duplicates. | High |
| FR-04 | System shall engineer 10 features: hour, day_of_week, month, is_weekend, day_of_year, lag_1h, lag_24h, lag_168h, rolling_3h, rolling_24h. | High |
| FR-05 | System shall predict average load for the next 24 hours using iterative Random Forest inference. | High |
| FR-06 | System shall predict hourly load for the next 168 hours (7 days) as an array. | High |
| FR-07 | System shall compare predicted 24-hour average against contract demand and return Safe or Risk status. | High |
| FR-08 | System shall display results: four KPI cards, load line chart, bar chart, forecast chart, alert card. | High |
| FR-09 | System shall store each prediction result in the database linked to the authenticated user. | Medium |

| | | |
|---|---|---|
| FR-10 | System shall display the 10 most recent predictions for the authenticated user. | Medium |
| FR-11 | System shall allow users to save factory settings including contract demand. | Medium |
| FR-12 | Saved contract demand shall auto-fill the dashboard input field on login. | Medium |
| FR-13 | All routes except sign-in and sign-up shall require a valid Clerk authentication session. | High |
| FR-14 | System shall rate-limit /predict to 10 requests per minute and read endpoints to 30 per minute per IP. | High |

## 2.3 Dependencies

The system depends on the following external services and libraries. Unavailability of any external service degrades or disables the corresponding function:

- Clerk (cloud SaaS) — User authentication. If Clerk is down, users cannot log in and the system is inaccessible.
- Neon PostgreSQL (cloud DBaaS) — Prediction storage and settings persistence. If unreachable, predictions still complete but are not saved. The system degrades gracefully with silent error logging.
- Railway (cloud PaaS) — Backend hosting. Downtime makes the prediction API unavailable.
- Vercel (cloud CDN) — Frontend hosting. Downtime makes the dashboard inaccessible.
- scikit-learn — RandomForestRegressor implementation. Version pinned in requirements.txt to ensure reproducibility.
- psycopg2-binary — PostgreSQL driver. Required for all database read and write operations.
- @clerk/nextjs — Clerk JavaScript SDK. Required for authentication integration in Next.js.
- Recharts — Chart library. Required for all dashboard visualizations.
- SlowAPI — Rate limiting middleware. Required for API abuse prevention.
- pandas and numpy — Data processing libraries. Required for feature engineering pipeline.

## 2.4 Performance Requirements

- /predict endpoint must return a response within 60 seconds for a standard 8,760-row CSV file on Railway free tier.
- Dashboard must load within 3 seconds on a standard broadband connection after successful authentication.
- Recent Predictions table must render within 2 seconds of page load under normal database response conditions.
- System must handle up to 50 concurrent authenticated users without response degradation.

- Cold start on Railway free tier (first request after idle period) adds 5–10 seconds and is acceptable given the non-real-time nature of usage.

## 2.5 Hardware Requirements

**Table 2.2: Hardware and Software Requirements**

| Component | Minimum Requirement |
|---|---|
| Development Machine | Windows 10/11, 8 GB RAM, 20 GB free disk space, Intel Core i5 or equivalent |
| Client Browser | Chrome 90+, Firefox 88+, Edge 90+, or Safari 14+ |
| Internet Connection | Required for all cloud services: Clerk, Neon, Railway, Vercel |
| Backend Server | Railway free tier: 1 vCPU, 512 MB RAM (sufficient for single-model inference) |
| Database | Neon free tier: cloud-hosted, no local hardware required |
| Python Version | 3.10 or above (project developed on Python 3.13) |
| Node.js Version | 18.0 or above |

## 2.6 Constraints & Assumptions

- The ML model is pre-trained and is not retrained at runtime. Prediction quality depends on the similarity between the user's uploaded data and the synthetic training distribution.
- Users must upload a minimum of 168 rows (one full week of hourly data) for the lag_168h feature to be computable.
- CSV files must contain exactly two columns named datetime (YYYY-MM-DD HH:MM:SS) and load_kw (numeric positive values).
- SCADA integration and real-time meter data streaming are out of scope for this version. Data ingestion is manual CSV upload only.
- Contract demand input must be between 1 and 100,000 kW. Values outside this range are rejected with an error message.
- It is assumed that uploaded data is hourly — one row per hour with no gaps. Sub-hourly or irregular intervals are not supported.
- It is assumed that Clerk, Neon, Railway, and Vercel are operational. No fallback hosting infrastructure exists in this version.
- The system does not provide legal or regulatory advice. Alert output is informational and should be used as a guide, not a guarantee.

# Chapter 3
## Design

### 3.1 Algorithm (if Applicable)

The core forecasting algorithm is a Random Forest Regressor with iterative multi-step prediction. The complete pipeline runs as follows:

1. Data Ingestion: Read the uploaded CSV file. Decode as UTF-8. Parse into a pandas DataFrame.
2. Validation: Check file extension (.csv only), size (≤ 5 MB), exactly two column names (datetime and load_kw), row count (168–10,000), numeric load values, and contract demand range (1–100,000 kW). Reject with error message on any failure.
3. Cleaning: Convert the datetime column to pandas datetime objects; drop rows where parsing fails. Remove rows with load_kw ≤ 0. Drop rows with duplicate datetime values. Sort by datetime ascending.
4. Feature Engineering: Compute hour, day_of_week, month, is_weekend, day_of_year from the datetime column. Compute lag_1h, lag_24h, lag_168h using pandas shift(). Compute rolling_3h and rolling_24h using pandas rolling().mean() with shift(1) to avoid data leakage. Drop NaN rows.
5. Prediction Window Setup: Take the last 170 rows as the initial window. Set prediction start time to datetime.now() — the current system time — not the last row timestamp in the uploaded file.
6. Iterative Inference: For each future hour i from 1 to H (H = 24 for next-day, H = 168 for next-week): append a row for time t+i with a NaN load value; compute all 10 features; fill any remaining NaN with 0; call model.predict() to get the load for that hour; store the prediction; write the predicted value back into the window for the next iteration.
7. Alert Logic: Compute next_day_avg = mean of the first 24 predicted values. If next_day_avg > contract_demand then status = 'Risk' else status = 'Safe'. Generate a descriptive alert message with exact values.
8. Storage and Response: Write the result to the Neon predictions table with the user's Clerk ID. If the database write fails, log the error silently and still return the prediction. Return PredictionResponse JSON to the frontend.

**Table 3.1: Ten Engineered Features with Type and Purpose**

| Feature | Type | pandas Operation | Purpose |
|---|---|---|---|
| hour | Time-based | dt.hour | Captures the daily load cycle |
| day_of_week | Time-based | dt.dayofweek | 0=Monday to 6=Sunday; weekly shift pattern |
| month | Time-based | dt.month | Seasonal variation (summer/winter peaks) |
| is_weekend | Time-based | (dow>=5).astype(int) | Binary factory shutdown flag |

| | | | |
|---|---|---|---|
| day_of_year | Time-based | dt.dayofyear | Annual load cycle (1–365) |
| lag_1h | Lag | load_kw.shift(1) | Load one hour ago |
| lag_24h | Lag | load_kw.shift(24) | Same hour yesterday |
| lag_168h | Lag | load_kw.shift(168) | Same hour last week (dominant feature) |
| rolling_3h | Rolling | load_kw.shift(1).rolling(3).mean() | 3-hour smoothed trend |
| rolling_24h | Rolling | load_kw.shift(1).rolling(24).mean() | 24-hour smoothed trend |

## 3.2 Function Oriented Design for Procedural Approach

The backend follows a procedural decomposition into five modules, each with a single responsibility. This separation makes each module independently testable and modifiable without affecting the others.

- main.py — Entry point. Initialises the FastAPI application, configures CORS middleware with the allowed origin from environment variables, loads the ML model files at server startup, configures SlowAPI rate limiting, and defines all five API endpoint functions (/health, /predict, /predictions, /settings GET, /settings POST).
- utils.py — Data utility functions. validate_csv(df) checks all format constraints and raises HTTP exceptions on failure. clean_data(df) removes invalid rows. add_features(df) computes all 10 engineered features and drops NaN rows. This module has no knowledge of the ML model or the database.
- model.py — Inference functions. load_model(model_path, features_path) loads the pkl files using joblib. predict_future(df, model, features, hours) runs the iterative prediction loop and returns a list of hourly forecast values. predict_next_24h() and predict_next_week() are thin wrappers that call predict_future with H=24 and H=168.
- database.py — Persistence functions. get_connection() opens a psycopg2 connection using the DATABASE_URL environment variable. save_prediction() inserts one row into the predictions table and commits. get_recent_predictions(user_id, limit) fetches the most recent N rows for a user ordered by created_at descending. get_user_settings(user_id) and save_user_settings(user_id, ...) handle the user_settings table with upsert logic.
- schemas.py — Response type definition. PredictionResponse is a Pydantic BaseModel with typed fields for status, alert_message, next_day_prediction (float), and next_week_prediction (list of floats). Pydantic validates the response before it is serialised to JSON.

## 3.3 System Design

### 3.3.1 Data Flow Diagrams (Level 0, Level 1)

Level 0 — Context Diagram: The entire system appears as a single process bubble. Two external entities interact with it: Factory Operator (provides CSV data and contract demand value; receives prediction results, Safe/Risk alert, and dashboard output) and Cloud Database

(receives stored prediction records and user settings; supplies prediction history and saved settings on request).

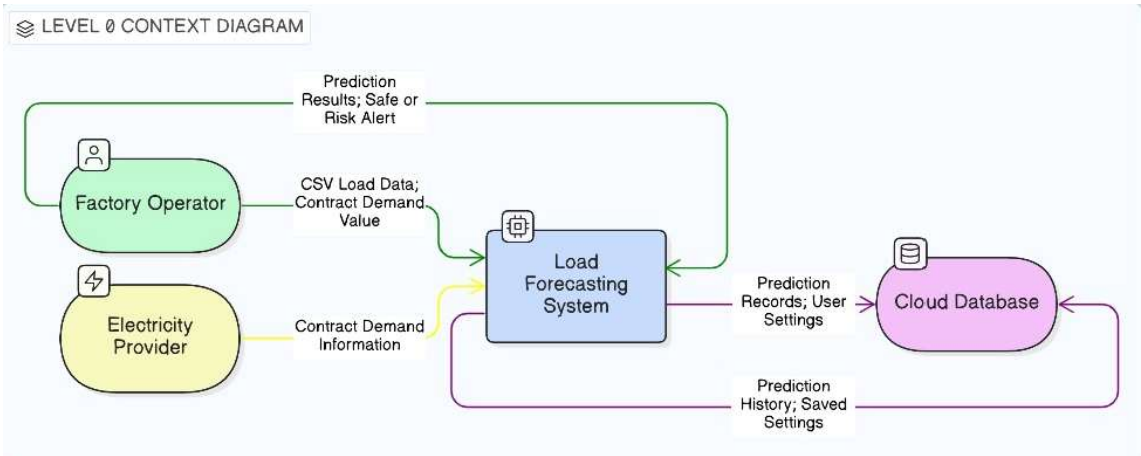



*Figure 3.1: Data Flow Diagram – Level 0 (Context Diagram)*

Level 1 — Detailed DFD: The system is decomposed into nine processes. Process 1.0 authenticates the user via Clerk. Process 2.0 validates and uploads the CSV file. Process 3.0 cleans and pre-processes the data. Process 4.0 engineers the 10 features. Process 5.0 runs the ML model prediction. Process 6.0 compares the predicted load against the contract demand. Process 7.0 generates the Safe or Risk alert. Process 8.0 stores the result in the database. Process 9.0 displays the dashboard to the user. Three data stores support the processes: D1 User Settings Store, D2 Predictions Store, and D3 ML Model Store (pkl files on Railway filesystem).

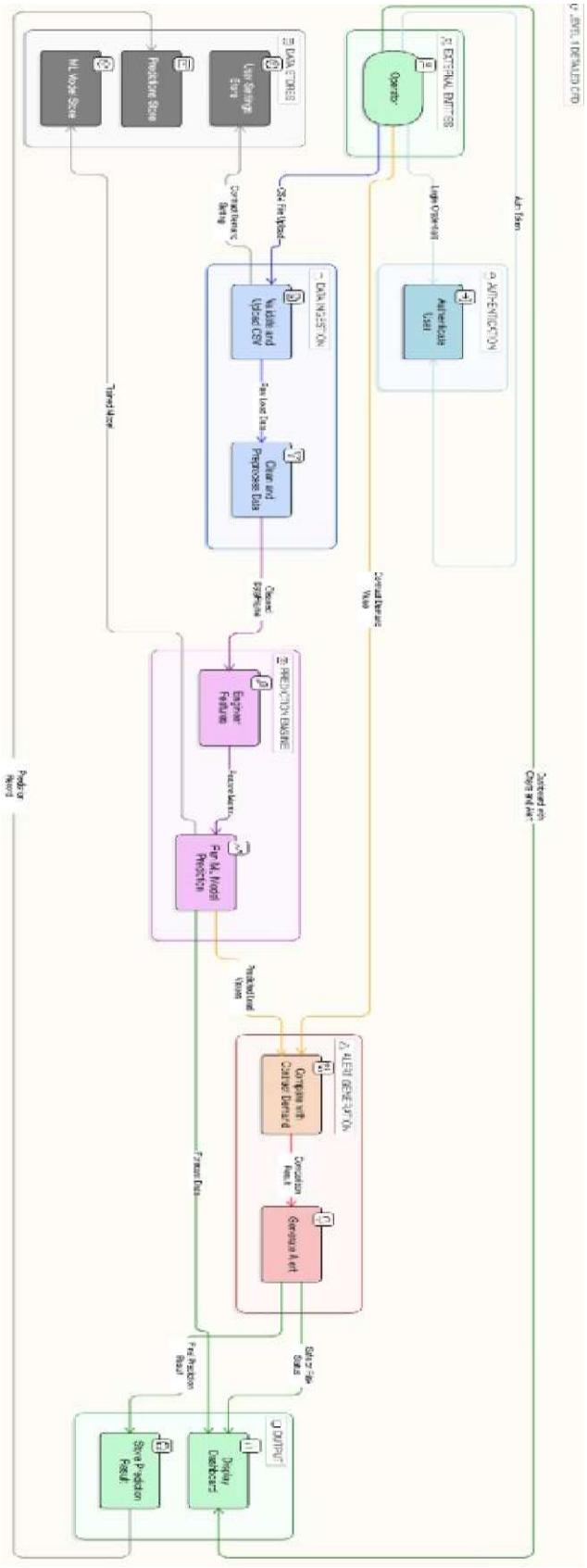


*Figure 3.2: Data Flow Diagram – Level 1 (Detailed)*

### 3.3.2 Activity Diagram

The Activity Diagram shows the complete control flow from user login through CSV upload, validation, feature engineering, model inference, alert evaluation, database write, and result display on the dashboard. Two decision nodes branch the flow: the CSV validation check (valid path continues; invalid path shows an error and returns the user to the upload step) and the contract demand comparison (load exceeding the limit takes the Risk branch; load within the limit takes the Safe branch). Both branches converge at the result storage and dashboard display state.

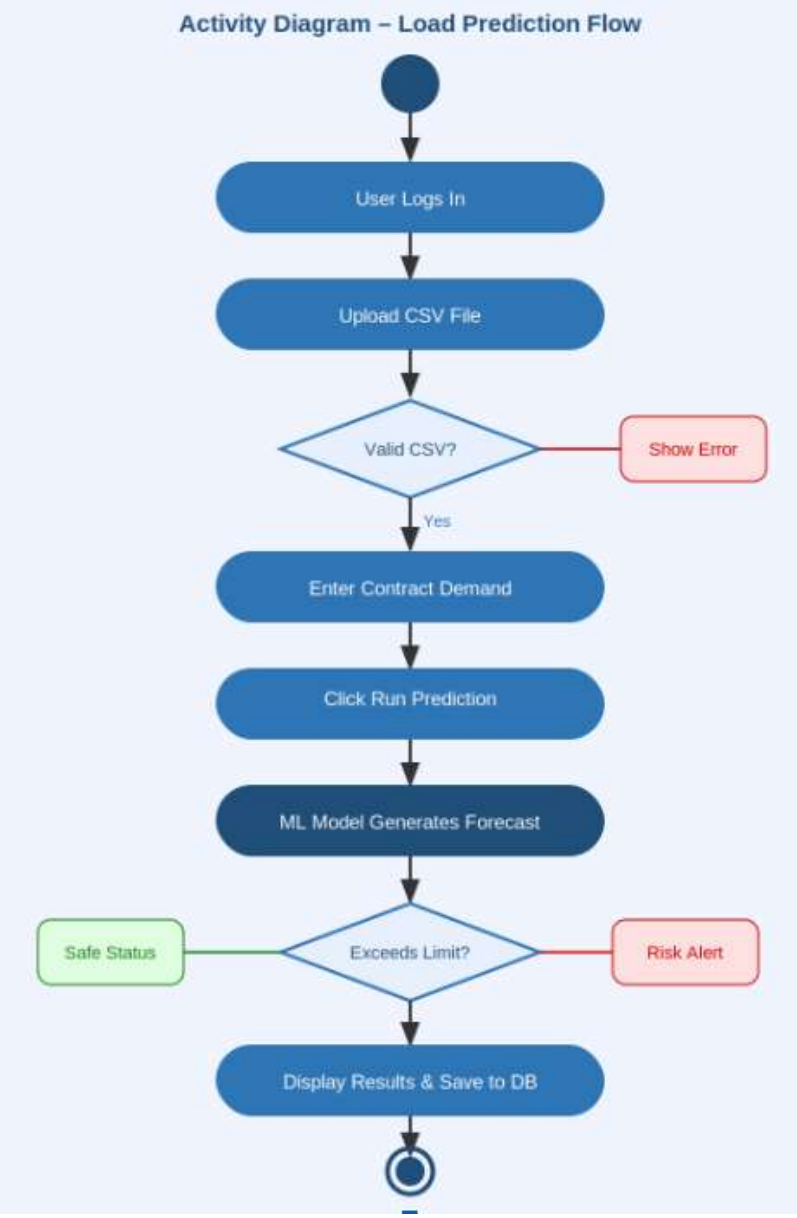



*Figure 3.3: Activity Diagram – Load Prediction Flow*

### 3.3.3 Flow Chart

The Flow Chart for the prediction process uses standard flowchart notation: ovals for start and end, rectangles for process steps, diamonds for decision points, and arrows for flow direction. The chart begins at the Run Prediction button click. It passes through file validation (two decision diamonds for format and size), proceeds to feature engineering and model inference, then to a decision diamond for the Safe/Risk threshold comparison. Two terminal paths lead to: prediction displayed with Safe alert, or prediction displayed with Risk alert. A separate error path from validation leads to an error display terminal.

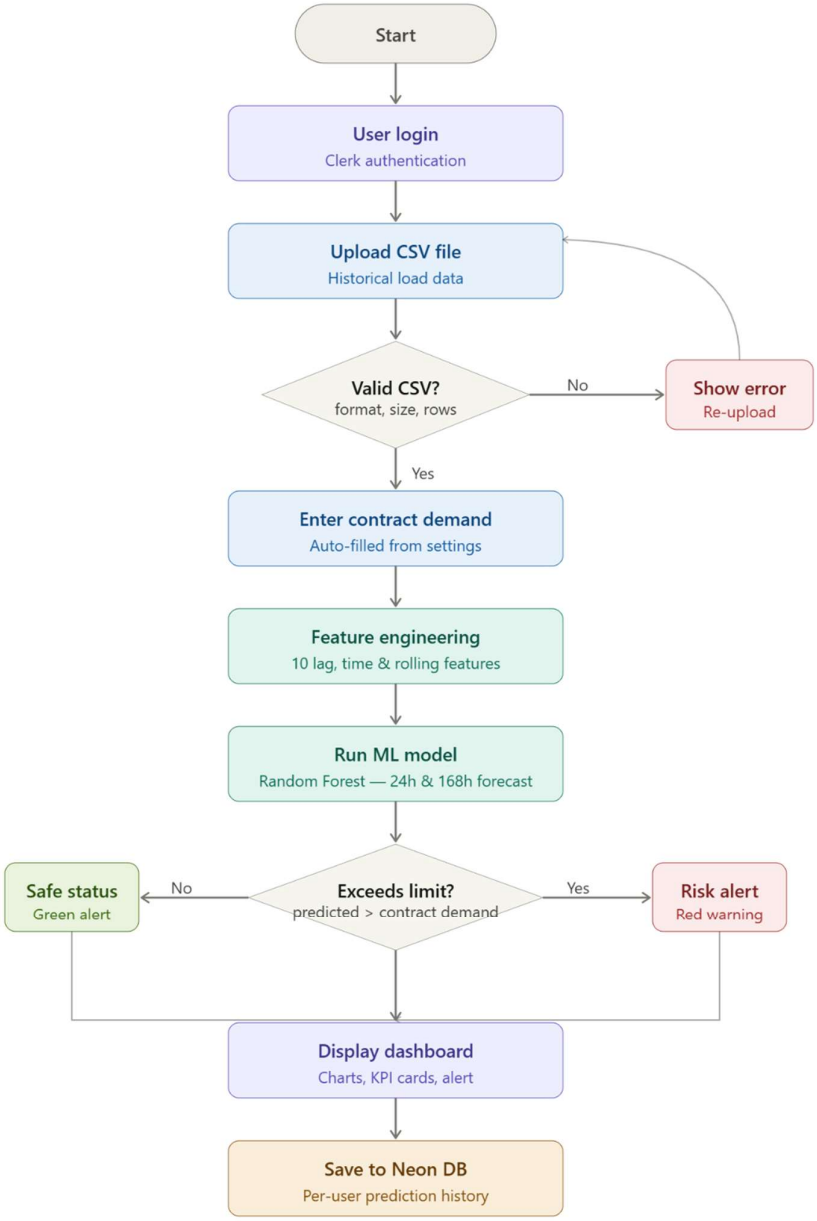



*Figure 3.4: Flow Chart – Prediction Process*

### 3.3.4 Class Diagram

The Class Diagram describes the static object structure using six principal classes. User holds userId, email, and createdAt attributes and a login() method. UserSettings holds userId, contractDemand, factoryName, industryType, and location with save() and load() methods; it has a one-to-one association with User. Prediction holds id, userId, createdAt, contractDemand, nextDayPrediction, nextWeekPrediction, status, and alertMessage with a save() method; it has a one-to-many association with User. MLModel holds modelPath, featuresPath, model, and features with load() and predictFuture() methods. DataProcessor holds df, minRows, and maxRows with validateCSV(), cleanData(), and addFeatures() methods. PredictionAPI holds allowedOrigin and rateLimit with methods for all five endpoints; it uses MLModel and DataProcessor and creates Prediction objects.

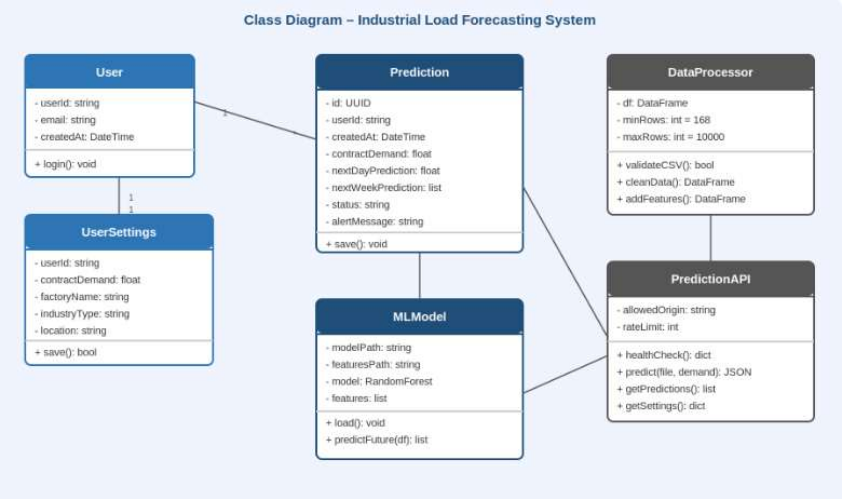



*Figure 3.5: Class Diagram – System Object Structure*

### 3.3.5 ER Diagram

The ER Diagram shows two database tables. The predictions table has primary key id (UUID) and foreign key user_id (VARCHAR) linking to the external Clerk user entity. It also holds created_at, contract_demand, next_day_prediction, next_week_prediction (JSONB), status, and alert_message. The user_settings table has primary key id (UUID) and a unique key user_id (VARCHAR); it holds contract_demand, factory_name, industry_type, location, created_at, and updated_at. The User entity (externally managed by Clerk, shown with a dashed border) has a one-to-one relationship with user_settings and a one-to-many relationship with predictions. Crow's foot notation is used for cardinality.

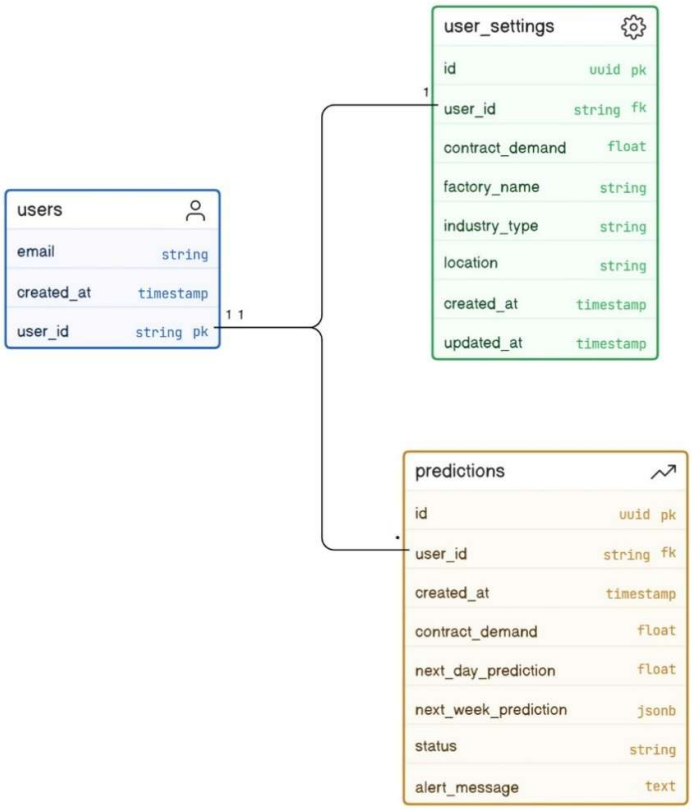



*Figure 3.6: ER Diagram – Database Tables and Relationships*

### 3.3.6 Sequence Diagram

The Sequence Diagram shows message passing between five lifelines: User, Frontend (React/Next.js), FastAPI Backend, ML Model, and Neon Database. The sequence is: (1) User uploads CSV and clicks Run Prediction; (2) Frontend sends POST /predict with FormData; (3) Backend validates, cleans, and engineers features; (4) Backend calls predictFuture() on the ML Model; (5) ML Model returns 24-hour and 168-hour forecast arrays; (6) Backend sends INSERT to Neon Database with prediction record and user_id; (7) Neon returns acknowledgement; (8) Backend returns PredictionResponse JSON; (9) Frontend updates all dashboard components reactively.

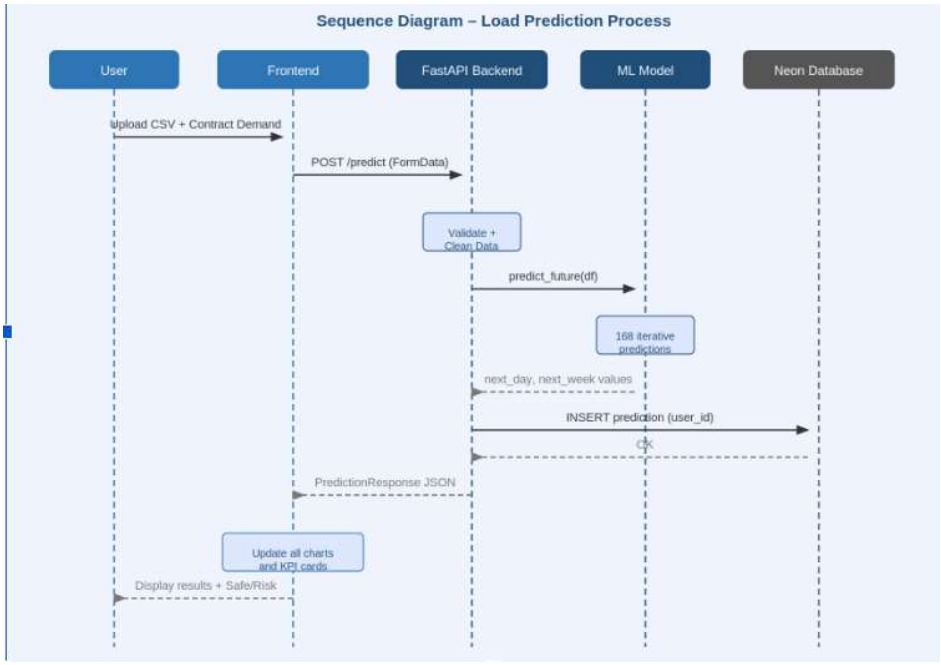



*Figure 3.7: Sequence Diagram – Load Prediction Process*

## 3.4 Database Design

### 3.4.1 Logical Database Design

Both tables satisfy Third Normal Form (3NF). In the predictions table, every non-key column depends directly on the UUID primary key with no transitive dependencies. In user_settings, user_id is a unique natural key; all other columns describe factory configuration attributes that depend solely on user_id and not on each other.

**Table 3.2: Logical Database Design – predictions Table**

| Column | Data Type | Constraint | Description |
|---|---|---|---|
| id | UUID | PK, DEFAULT gen_random_uuid() | Unique identifier for each prediction record |
| user_id | VARCHAR(255) | NOT NULL | Clerk user ID — links prediction to account |
| created_at | TIMESTAMP | DEFAULT NOW() | Timestamp when prediction was generated |
| contract_demand | FLOAT | NOT NULL | User-entered contract limit in kW |
| next_day_prediction | FLOAT | NOT NULL | 24-hour average predicted load in kW |
| next_week_prediction | JSONB | NOT NULL | Array of 168 hourly predicted load values |

| | | | |
|---|---|---|---|
| status | VARCHAR(10) | NOT NULL | 'Safe' if forecast ≤ demand; else 'Risk' |
| alert_message | TEXT | NOT NULL | Descriptive alert with exact load and limit values |

**Table 3.3: Logical Database Design – user_settings Table**

| Column | Data Type | Constraint | Description |
|---|---|---|---|
| id | UUID | PK, DEFAULT gen_random_uuid() | Unique identifier for settings record |
| user_id | VARCHAR(255) | UNIQUE, NOT NULL | Clerk user ID — one row per user (upsert enforced) |
| contract_demand | FLOAT | — | Saved contract limit — auto-fills dashboard |
| factory_name | VARCHAR(255) | — | Factory display name entered by user |
| industry_type | VARCHAR(255) | — | Industry category (Manufacturing, Textile, etc.) |
| location | VARCHAR(255) | — | City and state of the factory |
| created_at | TIMESTAMP | DEFAULT NOW() | When the settings record was first created |
| updated_at | TIMESTAMP | DEFAULT NOW() | When the settings were last saved |

### 3.4.2 Physical Database Design

Platform: Neon PostgreSQL (cloud-hosted, Singapore region). Connection method: psycopg2-binary Python driver using an SSL connection string stored in the DATABASE_URL environment variable on Railway. Neon's built-in pgBouncer connection pooler is used via the pooler endpoint to manage concurrent connections on the free tier.

The predictions table is append-only. Rows are inserted on each prediction run via save_prediction() and queried using SELECT ... WHERE user_id = %s ORDER BY created_at DESC LIMIT 10. No explicit indexes are created beyond the default primary key index; table size on the free tier stays well within the range where sequential scans are acceptable.

The user_settings table uses PostgreSQL upsert syntax: INSERT INTO user_settings (...) VALUES (...) ON CONFLICT (user_id) DO UPDATE SET ... This guarantees each user always has exactly one row regardless of how many times they click Save Settings. The updated_at column is explicitly set to NOW() on every upsert.

# Chapter 4
## Implementation, Testing, and Maintenance

### 4.1 Introduction to Languages, IDE's, Tools and Technologies used for Implementation

**Table 4.1: Technology Stack – Languages, IDEs, Tools and Technologies**

| Component | Technology / Tool | Version | Purpose |
|---|---|---|---|
| Frontend Language | TypeScript | 5.7.3 | Type-safe React component development |
| Frontend Framework | Next.js | 16.1.6 | React SSR, file-based routing, middleware |
| UI Library | React | 19.2.4 | Component-based dashboard UI |
| Styling | Tailwind CSS | v4.2.0 | Utility-first CSS styling |
| UI Components | shadcn/ui + Radix | Latest | Accessible, composable component primitives |
| Charts | Recharts | 2.15.0 | Line charts, bar charts, dashboard visualisations |
| Backend Language | Python | 3.13 | ML inference and API logic |
| Backend Framework | FastAPI | Latest | REST API with automatic OpenAPI docs and Pydantic validation |
| ASGI Server | Uvicorn | Latest | Production ASGI server for FastAPI |
| ML Library | scikit-learn | 1.6.1 | RandomForestRegressor training and inference |
| Data Processing | pandas + numpy | Latest | Feature engineering, data cleaning |
| Model Persistence | joblib | Latest | Serialise and load .pkl model files |
| Rate Limiting | SlowAPI | Latest | IP-based rate limiting middleware |
| Env Management | python-dotenv | Latest | Load secrets from .env into environment |
| Database Driver | psycopg2-binary | Latest | Python PostgreSQL connection driver |
| Auth (Frontend) | @clerk/nextjs | Latest | Clerk SDK for Next.js authentication |
| Database | Neon PostgreSQL | Cloud | Cloud-hosted relational DB for predictions and settings |
| Auth Service | Clerk | Cloud | User signup, login, JWT session tokens |
| Frontend Hosting | Vercel | Cloud | Next.js build deployment via CDN |
| Backend Hosting | Railway | Cloud | Python FastAPI deployment (PaaS) |

| | | | |
|---|---|---|---|
| Version Control | Git + GitHub | Latest | Source control; URL: github.com/ashika1jain/industrial_load_dashboard- |
| IDE | VS Code | Latest | All development on Windows 11 |
| ML Notebook | Jupyter Notebook | Latest | Model training, feature analysis, visualisation |

## 4.2 Testing Techniques and Test Plans (According to Project)

Black-box functional testing was applied to all API endpoints and frontend workflows. Each test specifies the input given, the expected output defined by requirements, and the actual output observed during testing.

**Table 4.2: API Endpoints – Method, Input, Output, Rate Limit**

| Endpoint | Method | Input | Output | Rate Limit |
|---|---|---|---|---|
| /health | GET | None | { status: 'ok' } | 30/min |
| /predict | POST | CSV file + contract_demand + user_id (form) | PredictionResponse JSON | 10/min |
| /predictions | GET | user_id (query param) | List of prediction records | 30/min |
| /settings | GET | user_id (query param) | UserSettings JSON | 30/min |
| /settings | POST | user_id, demand, factory details (form) | { success: bool } | 10/min |

**Table 4.3: Test Cases and Results**

| Test Case | Input | Expected Result | Actual Result | Status |
|---|---|---|---|---|
| Valid CSV upload | 8760-row CSV | File accepted, green status | File accepted, filename shown green | Pass |
| Wrong file type | .xlsx file | Error: only .csv allowed | Error message shown | Pass |
| Too few rows | 30-row CSV | Error: minimum 168 rows | Error message shown | Pass |
| File over 5 MB | >5 MB CSV | Error: maximum 5 MB | Error message shown | Pass |
| Wrong column names | 'date','power' CSV | Error: invalid column names | Error message shown | Pass |
| Prediction — Safe | CSV + demand 400 | Status: Safe with exact values | Safe status, correct alert message | Pass |
| Prediction — Risk | CSV + demand 150 | Status: Risk, red alert | Risk status, red alert displayed | Pass |

| | | | | |
|---|---|---|---|---|
| Settings save + auto-fill | Save demand 300, reload | Demand field shows 300 on reload | Field auto-filled with 300 | Pass |
| Rate limit | 11 POST /predict in 60 sec | 429 on 11th request | 429 Too Many Requests returned | Pass |
| Auth redirect | Access without Clerk session | Redirect to /sign-in | Redirect occurs correctly | Pass |
| DB unavailable fallback | Predict with Neon unreachable | Prediction returns; error logged | Result returned; error in logs | Pass |
| User data isolation | Two accounts each predict | Each sees only their own history | Tables show own records only | Pass |

## 4.3 Installation Instructions

Prerequisites: Node.js 18+, Python 3.10+, Git, a Clerk account (clerk.com), a Neon account (neon.tech).

Backend Setup:

1. Clone the repository: git clone https://github.com/ashika1jain/industrial_load_dashboard-
2. Navigate to backend folder: cd backend
3. Install Python dependencies: pip install -r requirements.txt --break-system-packages
4. Create a .env file with two values: ALLOWED_ORIGIN=http://localhost:3000 and DATABASE_URL=[paste your Neon connection string here]
5. On Windows PowerShell, add Python Scripts to PATH: $env:PATH += ';C:\Users\hp\AppData\Roaming\Python\Python313\Scripts'
6. Start the backend server: uvicorn main:app --reload --port 8000

Frontend Setup:

1. Navigate to frontend folder: cd frontend
2. Install Node dependencies: npm install
3. Create a .env.local file with: NEXT_PUBLIC_API_URL=http://localhost:8000 and NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY=[your Clerk publishable key]
4. Start the development server: npm run dev
5. Open a browser and go to: http://localhost:3000

## 4.4 End User Instructions

1. Open the application URL in any modern browser (Chrome, Firefox, Edge, Safari).
2. Sign in with your email address or Google account through the Clerk login page.

3. On first login, click the settings gear icon in the top navigation bar. Enter your factory name, select industry type, enter your location, and set your contract demand in kilowatts. Click Save Settings.
4. Return to the main dashboard. The contract demand field will have auto-filled from your saved settings.
5. Click Upload CSV File or drag and drop your historical electricity load data file. The file must have two columns: datetime (YYYY-MM-DD HH:MM:SS) and load_kw (positive numbers). Minimum 168 rows required.
6. Check the contract demand input field and adjust if needed for this specific prediction run.
7. Click Run Prediction. Wait approximately 45–60 seconds for the results to load.
8. Read the Smart Alert card: a green card with a checkmark means Safe — your predicted load is within the contract limit. A red card with a warning triangle means Risk — take preventive action.
9. Review the Load Analytics chart (next 24 hours) and the 7-Day Forecast chart. The red dashed line on both charts shows your contract demand limit.
10. Scroll down to the Recent Predictions table to review your prediction history from previous sessions.

# Chapter 5
## Results and Discussions

### 5.1 User Interface Representation (of Respective Project)

The user interface is a single-page dashboard built with React 19 and Next.js 16. It is organized into eight vertical sections: a top navigation bar, four KPI cards, data upload and contract demand input panels, a 24-hour load analytics chart, a prediction results and alert panel, a 7-day forecast section, and a recent predictions table at the bottom.

The design uses Tailwind CSS with shadcn/ui component primitives. The color scheme uses blue tones for Safe states and red for Risk states. All charts are built with Recharts and include a red dashed reference line drawn at the contract demand level so users can immediately see the relationship between predicted load and the penalty threshold. The interface is fully responsive and functions on desktop and tablet browsers without horizontal scrolling.

The top navigation bar contains the system branding on the left, a notification bell icon, a settings gear icon that navigates to the Settings page, and the Clerk UserButton on the right for profile access and logout. The Settings page has four tabs: Factory Settings (active), Alert Preferences, Integrations, and Security. Factory Settings allows users to save factory name, industry type, location, and contract demand. The other three tabs are reserved for future functionality.

### 5.2 Brief Description of Various Modules of the System

**Table 5.1: Module Descriptions**

| Module | File / Location | Description |
|---|---|---|
| Main Dashboard | app/page.tsx | Manages global state (file, predictionData, isLoading, error, contractDemand). Orchestrates all child components and calls lib/api.ts on prediction run. |
| Top Navigation | components/top-nav.tsx | Navigation bar with Clerk UserButton, notification bell, and settings gear icon linking to /settings page. |
| KPI Card | components/kpi-card.tsx | Reusable metric card. Four instances display: current load, next-day predicted load, weekly average predicted load, and contract demand. |
| Data Upload Section | components/data-upload... | Drag-and-drop and file-select CSV upload. Validates file extension client-side. Shows filename and upload status. |
| Demand Control Panel | components/demand-ctrl... | Contract demand numeric input with auto-fill from saved settings. Run Prediction button with loading spinner during inference. |

| | | |
|---|---|---|
| Load Chart | components/load-chart.tsx | 24-hour predicted load line chart with red dashed contract limit reference line using Recharts LineChart. |
| Prediction Results | components/pred-results.tsx | Next-day value, Safe/Risk badge, 7-day daily average bar chart (red bars for days above limit), and contract margin percentage. |
| Alert Panel | components/alert-panel.tsx | Smart Alert card: green CheckCircle icon for Safe; red AlertTriangle icon for Risk. Shows exact predicted load and limit values in the message. |
| Forecast Section | components/forecast-sec.tsx | Two summary cards (next-day and 7-day average) and a weekly line chart with contract limit reference line. |
| Recent Predictions Table | components/recent-pred.tsx | Fetches the last 10 predictions for the authenticated user from GET /predictions and renders them with Safe/Risk badges and timestamps. |
| Settings Page | app/settings/page.tsx | Tabbed settings page. Factory Settings tab is active and saves data to Neon via POST /settings. Other tabs are future scope placeholders. |
| API Utility | lib/api.ts | Client-side API functions: runPrediction(), getRecentPredictions(), getUserSettings(), saveUserSettings(). All calls include the Clerk user ID. |
| Backend Entry Point | backend/main.py | FastAPI application. Loads ML model at startup. Defines /health, /predict, /predictions, /settings endpoints. Configures CORS and rate limiting. |
| ML Inference | backend/model.py | load_model() loads pkl files. predict_future() runs iterative inference. Wrappers: predict_next_24h() and predict_next_week(). |
| Data Processing | backend/utils.py | validate_csv(), clean_data(), add_features() — the full data preparation pipeline for both training-compatible feature engineering and cleaning. |
| Database Access | backend/database.py | save_prediction(), get_recent_predictions(), get_user_settings(), save_user_settings() using psycopg2 with upsert for settings. |
| Response Schema | backend/schemas.py | PredictionResponse Pydantic model defining typed API response structure: status, alert_message, next_day_prediction (float), next_week_prediction (list). |

## 5.3 Snapshots of System with Brief Detail of Each

The following screenshots show the working system. Attach actual screenshots of the deployed application before final submission.

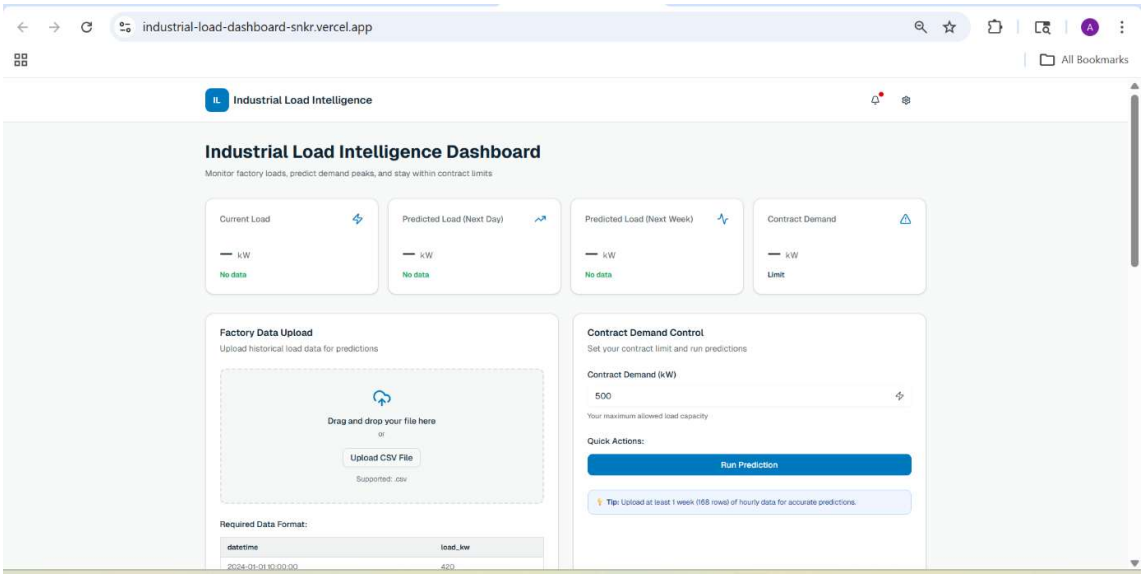



*Figure 5.1: Dashboard – Initial State. All KPI cards show dashes. Upload and demand panels are ready. Charts show placeholder text.*

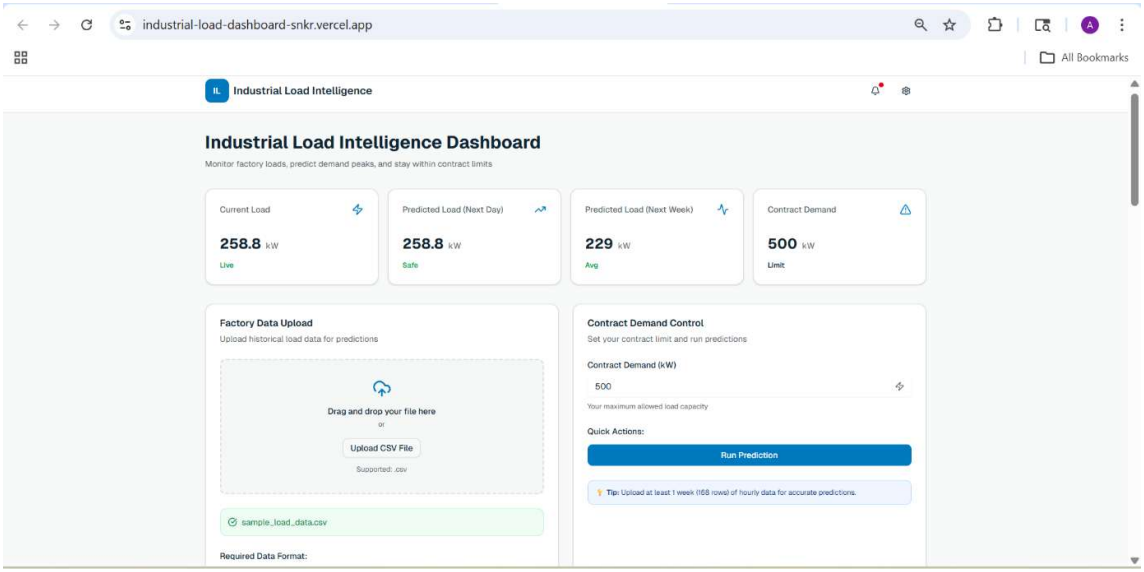



*Figure 5.2: Dashboard – After Prediction (Safe). Smart Alert card shows green checkmark. Load chart shows forecast below the red dashed contract limit line.*

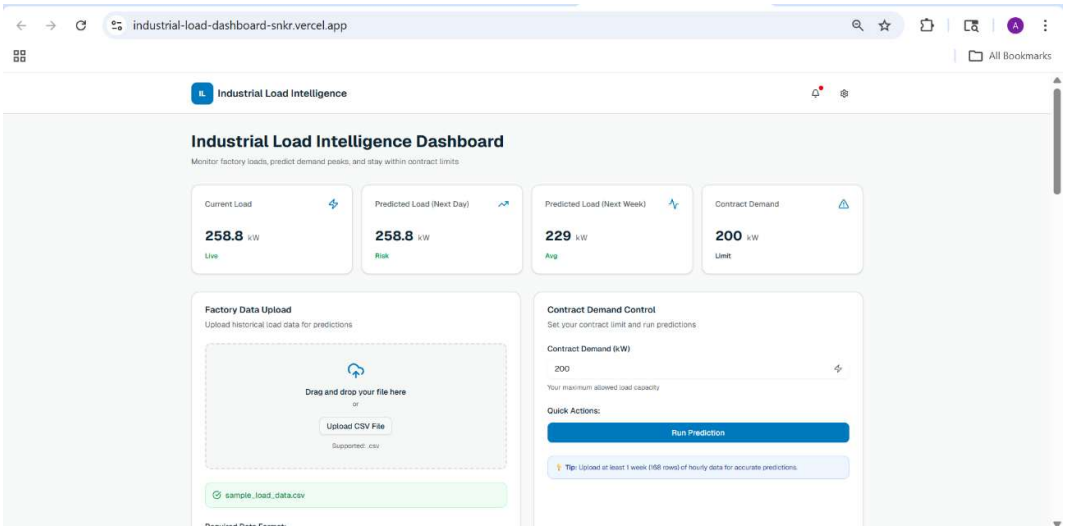



*Figure 5.3: Dashboard – After Prediction (Risk). Smart Alert card shows red warning triangle. Contract demand is set below predicted load. Bar chart shows red bars for days above the limit.*

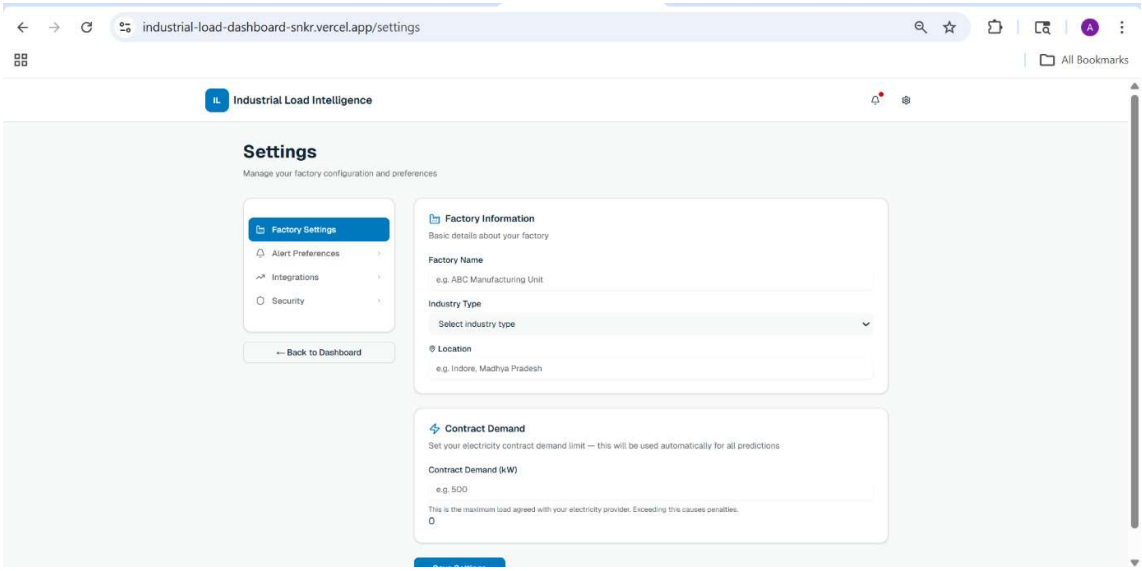



*Figure 5.4: Settings Page – Factory Settings Tab. Factory name, industry type, location, and contract demand fields visible with Save Settings button.*

## 5.4 Back Ends Representation (Database to be used)

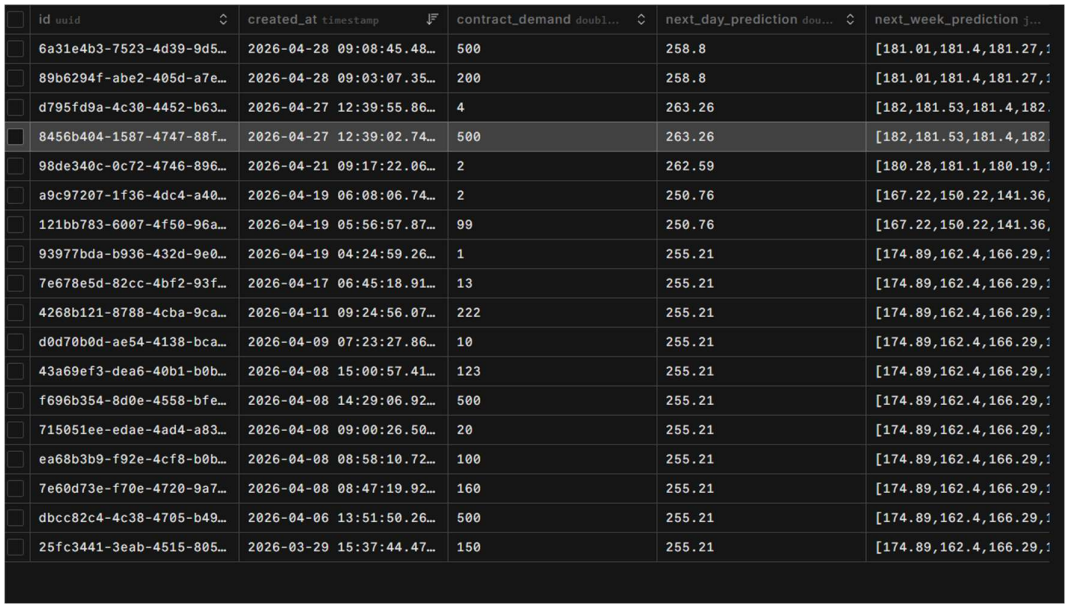
The backend uses Neon PostgreSQL as the cloud database. Two tables are hosted in the production branch of the Neon project named 'industrial-load-dashboard'. The backend connects using psycopg2 with an SSL connection string stored in the DATABASE_URL environment variable on Railway. The Neon web console provides a Tables view, SQL Editor, and monitoring dashboard for inspecting data and running queries.

All database writes use a connection-per-request pattern: a psycopg2 connection is opened, the query is executed, committed, and the connection is closed. Neon's built-in pgBouncer pooler handles concurrent connection requests on the free tier without requiring a persistent connection pool in application code.

The predictions table is insert-only. Each call to POST /predict adds one row. The user_settings table uses upsert semantics so each user always has exactly one row. Both tables are queried using parameterized queries to prevent SQL injection.

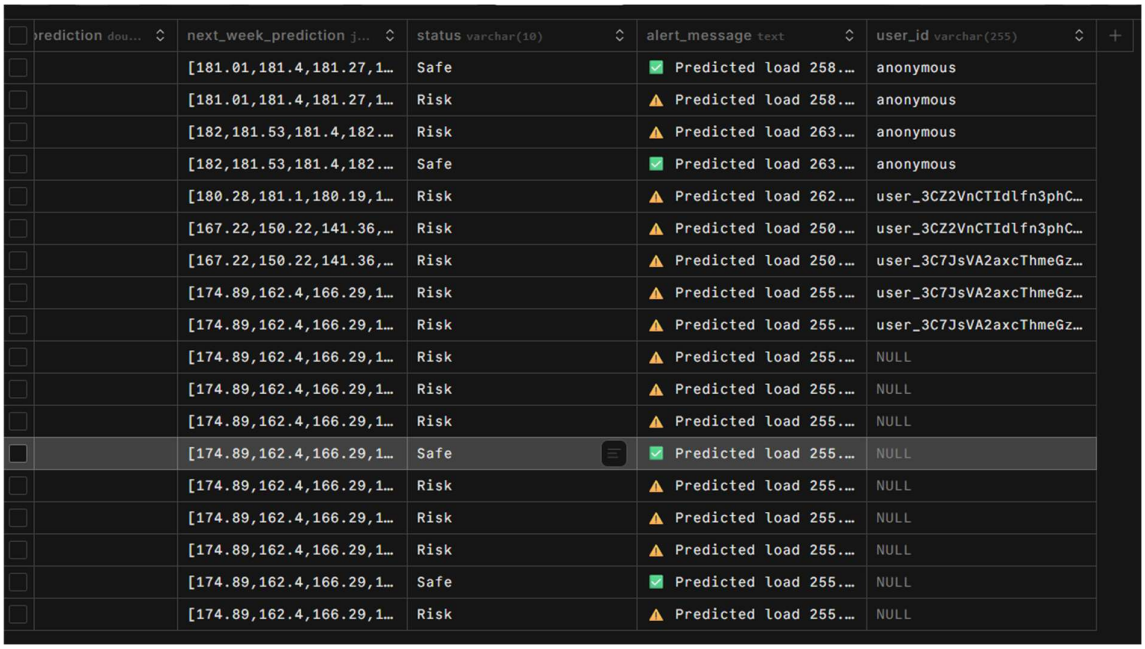
## 5.5 Snapshots of Database Tables with Brief Description

The following screenshots show the Neon database tables. Attach actual Neon console screenshots before final submission.



| id uuid | created_at timestamp | contract_demand doubl... | next_day_prediction dou... | next_week_prediction j... |
|---|---|---|---|---|
| 6a31e4b3-7523-4d39-9d5… | 2026-04-28 09:08:45.48… | 500 | 258.8 | [181.01,181.4,181.27,1 |
| 89b6294f-abe2-405d-a7e… | 2026-04-28 09:03:07.35… | 200 | 258.8 | [181.01,181.4,181.27,1 |
| d795fd9a-4c30-4452-b63… | 2026-04-27 12:39:55.86… | 4 | 263.26 | [182,181.53,181.4,182 |
| 8456b404-1587-4747-88f… | 2026-04-27 12:39:02.74… | 500 | 263.26 | [182,181.53,181.4,182 |
| 98de340c-0c72-4746-896… | 2026-04-21 09:17:22.06… | 2 | 262.59 | [180.28,181.1,180.19,1 |
| a9c97207-1f36-4dc4-a40… | 2026-04-19 06:08:06.74… | 2 | 250.76 | [167.22,150.22,141.36, |
| 121bb783-6007-4f50-96a… | 2026-04-19 05:56:57.87… | 99 | 250.76 | [167.22,150.22,141.36, |
| 93977bda-b936-432d-9e0… | 2026-04-19 04:24:59.26… | 1 | 255.21 | [174.89,162.4,166.29,1 |
| 7e678e5d-82cc-4bf2-93f… | 2026-04-17 06:45:18.91… | 13 | 255.21 | [174.89,162.4,166.29,1 |
| 4268b121-8788-4cba-9ca… | 2026-04-11 09:24:56.07… | 222 | 255.21 | [174.89,162.4,166.29,1 |
| d0d70b0d-ae54-4138-bca… | 2026-04-09 07:23:27.86… | 10 | 255.21 | [174.89,162.4,166.29,1 |
| 43a69ef3-dea6-40b1-b0b… | 2026-04-08 15:00:57.41… | 123 | 255.21 | [174.89,162.4,166.29,1 |
| f696b354-8d0e-4558-bfe… | 2026-04-08 14:29:06.92… | 500 | 255.21 | [174.89,162.4,166.29,1 |
| 715051ee-edae-4ad4-a83… | 2026-04-08 09:00:26.50… | 20 | 255.21 | [174.89,162.4,166.29,1 |
| ea68b3b9-f92e-4cf8-b0b… | 2026-04-08 08:58:10.72… | 100 | 255.21 | [174.89,162.4,166.29,1 |
| 7e60d73e-f70e-4720-9a7… | 2026-04-08 08:47:19.92… | 160 | 255.21 | [174.89,162.4,166.29,1 |
| dbcc82c4-4c38-4705-b49… | 2026-04-06 13:51:50.26… | 500 | 255.21 | [174.89,162.4,166.29,1 |
| 25fc3441-3eab-4515-805… | 2026-03-29 15:37:44.47… | 150 | 255.21 | [174.89,162.4,166.29,1 |

*Figure 5.5: Neon Database – predictions Table. Columns visible: id (UUID), user_id (Clerk ID), created_at (timestamp), contract_demand, next_day_prediction, status. The next_week_prediction column (JSONB array of 168 values) is visible in the SQL Editor view.*



| prediction dou... | next_week_prediction j... | status varchar(10) | alert_message text | user_id varchar(255) |
|---|---|---|---|---|
| | [181.01,181.4,181.27,1… | Safe | Predicted load 258.… | anonymous |
| | [181.01,181.4,181.27,1… | Risk | Predicted load 258.… | anonymous |
| | [182,181.53,181.4,182.… | Risk | Predicted load 263.… | anonymous |
| | [182,181.53,181.4,182.… | Safe | Predicted load 263.… | anonymous |
| | [180.28,181.1,180.19,1… | Risk | Predicted load 262.… | user_3CZ2VnCTIdlfn3phC… |
| | [167.22,150.22,141.36,… | Risk | Predicted load 250.… | user_3CZ2VnCTIdlfn3phC… |
| | [167.22,150.22,141.36,… | Risk | Predicted load 250.… | user_3C7JsVA2axcThmeGz… |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | user_3C7JsVA2axcThmeGz… |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | user_3C7JsVA2axcThmeGz… |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | NULL |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | NULL |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | NULL |
| | [174.89,162.4,166.29,1… | Safe | Predicted load 255.… | NULL |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | NULL |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | NULL |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | NULL |
| | [174.89,162.4,166.29,1… | Safe | Predicted load 255.… | NULL |
| | [174.89,162.4,166.29,1… | Risk | Predicted load 255.… | NULL |

*Figure 5.6: Neon Database – user_settings Table. One row per Clerk user. Columns include user_id (unique), contract_demand, factory_name, industry_type, location, created_at, updated_at. Upsert behavior confirmed by single row per account.*

# Chapter 6
## Summary and Conclusions

This project designed, built, and deployed a complete web-based Industrial Load Forecasting and Contract Demand Alert System. The work covered six areas: synthetic data generation and feature engineering, Random Forest model training and evaluation, FastAPI backend development with full security hardening, React and Next.js frontend dashboard development, Neon PostgreSQL database integration for per-user prediction history, and Clerk-based authentication with factory settings persistence.

The Random Forest model trained on one year of synthetic hourly factory data with ten engineered features achieves a Mean Absolute Error of 15.44 kW, Root Mean Square Error of 19.44 kW, and Mean Absolute Percentage Error of 8.09% on a time-ordered held-out test set — corresponding to approximately 92% prediction accuracy. The most important feature is lag_168h (same hour, same weekday, previous week) with a Gini importance score of 0.963, confirming that weekly periodicity is the dominant pattern in factory load behavior.

**Table 6.1: Model Evaluation Metrics**

| Metric | Value | Interpretation |
|---|---|---|
| MAE | 15.44 kW | Average absolute prediction error. For a 500 kW limit, this is ~3% of the threshold. |
| RMSE | 19.44 kW | Root mean square error. Few large outlier errors; most predictions within 15–20 kW. |
| MAPE | 8.09% | Mean absolute percentage error. Approximately 92% overall accuracy on the test set. |

All 12 functional test cases passed, including the database unavailability fallback — a real-world condition discovered during development when ISP-level blocking prevented the backend from reaching the Neon pooler endpoint. The graceful fallback ensures the prediction result is still returned to the user even when the database write fails, with the error logged silently rather than surfaced as a 500 response.

The system demonstrates that lightweight, CSV-based industrial load forecasting is practical and deployable for small and mid-sized factories without hardware integration, SCADA access, or enterprise software subscriptions. A factory operator with a browser and a spreadsheet of meter readings can access full forecasting and alerting capability in under two minutes.

# Chapter 7
## Future Scope

- SCADA and smart meter integration: Direct connection to factory data sources would replace manual CSV upload with continuous automatic data ingestion, enabling real-time monitoring rather than periodic batch analysis.

- Email and SMS alerts: Automated notifications triggered by a Risk prediction would allow operators to act without needing to check the dashboard. Notification provider selection, message templates, and trigger conditions would need to be designed.

- Automated model retraining: A pipeline that retrains the model on real factory data uploaded by users would allow forecasts to be personalized to each facility's specific load signature over time. This would require a versioning strategy and rollback mechanism.

- Raw material pricing integration: Incorporating commodity price data as additional input features could improve forecast accuracy for factories whose production schedules are price-sensitive (steel, cotton, cement manufacturing).

- Multi-factory account support: Allowing a single account to manage multiple factory units — each with separate historical data, contract limits, and prediction histories — would make the system suitable for industrial groups operating several plants.

- Weather data integration: Temperature and humidity data from a weather API would help the model account for cooling and heating loads that drive seasonal demand variation, particularly for climate-sensitive production facilities.

- Prediction confidence intervals: Displaying uncertainty bands around the 168-hour forecast using quantile regression or bootstrap prediction intervals would give operators clearer guidance on how much weight to place on a Risk status in the later days of the forecast.

- Mobile application: A Progressive Web App or React Native version would allow factory floor supervisors to check alerts from a mobile device without requiring a desktop browser.

# Appendix

## Appendix I — CSV Data Format Specification

The system accepts CSV files with exactly two columns in the following format:

| Column Name | Data Type | Required Format | Example |
|---|---|---|---|
| datetime | String / DateTime | YYYY-MM-DD HH:MM:SS (hourly intervals) | 2024-01-15 09:00:00 |
| load_kw | Float | Positive numeric value | 342.7 |

Constraints: minimum 168 rows, maximum 10,000 rows, file size under 5 MB, no additional columns, no missing datetime values, all load_kw values must be positive numbers greater than zero.

## Appendix II — Model Feature Order

The trained model requires exactly these 10 features in this exact sequence: hour, day_of_week, month, is_weekend, day_of_year, lag_1h, lag_24h, lag_168h, rolling_3h, rolling_24h. This order is persisted in model_features.pkl and loaded at backend startup to guarantee consistency between training and inference. Any deviation in column order will produce incorrect predictions.

# Bibliography


[1] G. E. P. Box, G. M. Jenkins, G. C. Reinsel, and G. M. Ljung, Time Series Analysis: Forecasting and Control, 5th ed. Hoboken, NJ: Wiley, 2015.

[2] E. A. Feinberg and D. Genethliou, "Load forecasting," in Applied Mathematics for Restructured Electric Power Systems, J. H. Chow, F. F. Wu, and J. A. Momoh, Eds. Boston, MA: Springer, 2005, pp. 269-285.

[3] G. K. F. Tso and K. K. W. Yau, "Predicting electricity energy consumption: A comparison of regression analysis, decision tree and neural networks," Energy, vol. 32, no. 9, pp. 1761-1768, Sep. 2007.

[4] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5-32, Oct. 2001.

[5] A. S. Ahmad et al., "A review on applications of ANN and SVM for building electrical energy consumption forecasting," Renewable and Sustainable Energy Reviews, vol. 33, pp. 102-109, May 2014.

[6] X. Qiu, P. N. Suganthan, and G. A. J. Amaratunga, "Ensemble incremental learning Random Forest for stream learning and concept drift detection," Soft Computing, vol. 22, no. 15, pp. 5103-5123, Aug. 2018.

[7] V. Marinakis et al., "From big data to smart energy services: An application for intelligent energy management," Future Generation Computer Systems, vol. 110, pp. 572-586, Sep. 2020.

[8] S. Ramirez, FastAPI Documentation, 2024. [Online]. Available: https://fastapi.tiangolo.com. Accessed: Apr. 20, 2026.

[9] Vercel Inc., Next.js Documentation, 2024. [Online]. Available: https://nextjs.org/docs. Accessed: Apr. 20, 2026.

[10] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

# List of Publications (If any)

No publications have been submitted or accepted in connection with this project work at the time of report submission. The work is original and has not been published elsewhere.

# Reprints of Publications (If any)

Not applicable. No publications exist in connection with this project at the time of submission.